Manual del Programador: Arquitectura Orientada a Objetos y Motor de Juego en C++ (class Nave)

Luca Di Guglielmo

Laboratorio de Programacion, Escuela de Educacion Secundaria Tecnica Nro. 1 "Eduardo Ader"

Curso: 5to Año 3ra División - Grupos A y B

Profesor: Mansilla Munoz York Elias & Yamil Ganduglia

21 de octubre de 2026

Introducción

El presente manual técnico de programación tiene como propósito fundamental documentar la arquitectura de software, las decisiones de diseño orientado a objetos y los patrones de gestión de memoria implementados en el desarrollo del motor de juego espacial en consola de comandos C + +. El proyecto constituye el eje práctico de la Actividad 8 de la asignatura Laboratorio de Programación (LPR), cursada en el 5to año de la Escuela de Educación Secundaria Tecnica Nro. 1 "Eduardo Ader" durante el ciclo lectivo 2026. Esta guía técnica se encuentra estrictamente adaptada a los estándares APA v7

En el desarrollo de software industrial contemporáneo, la transición desde el paradigma de programacion estructurada hacia el paradigma de la Programación Orientada a Objetos (POO) representa un salto cualitativo indispensable. Mientras que en la Actividad 7 se emplearon estructuras (struct) como simples paquetes pasivos de datos con funciones dispersas en el ámbito global, la presente actividad adopta formalmente el concepto de clase (class). Esta transición incorpora dos pilares decisivos de la ingeniería de software: la privacidad de datos mediante encapsulamiento estricto (sección private:), que salvaguarda el estado interno del objeto frente a mutaciones accidentales, y la definicion de metodos y comportamientos cohesivos (sección public:), que otorgan al objeto plena autonomía operativa.

Desarrollo

Esta sección desglosa minuciosamente el entorno de compilación, las rutinas de bajo nivel para la manipulación de la consola de Windows, la arquitectura interna de la clase Nave y sus entidades satelitales, los algoritmos de gestión dinámica de memoria y la estructuración del repositorio institucional de control de versiones.

Entorno Técnico de Desarrollo y Bibliotecas del Sistema

El código fuente fue concebido para ser compilado bajo el estándar C++11 o superior mediante la cadena de herramientas GNU Compiler Collection (GCC / MinGW), ejecutándose de forma nativa sobre el subsistema de consola de Microsoft Windows. El proyecto articula un conjunto seleccionado de bibliotecas estándar y de bajo nivel:

1. iostream: Canaliza los flujos estandar de entrada y salida de datos (cout y cin) para la impresion de caracteres y solicitud inicial de credenciales.

2. windows.h: Provee acceso a la API del sistema operativo Windows, habilitando los identificadores de salida de consola (HANDLE), el posicionamiento cartesiano del cursor y la modulacion de intervalos temporales mediante Sleep().

3. conio.h y cctype: Proveen las funciones _kbhit() y _getch() para capturar pulsaciones de teclado de forma no bloqueante, complementadas con tolower() para normalizar entradas de texto.

4. vector y ctime: Habilitan la generacion y administracion dinamica de colecciones de punteros a objetos en tiempo de ejecucion y la inicializacion pseudoaleatoria de semillas temporales mediante srand(time(NULL)).

Rutinas de Bajo Nivel y Manipulacion de la Consola de Windows

Para lograr un motor grafico en modo texto con altas tasas de refresco y libre de

parpadeos intrusivos, se desarrollaron dos funciones auxiliares criticas que interactuan directamente con la interfaz del subsistema Win32:

La rutina gotoxy(int x, int y) obtiene el manejador de la consola estandar mediante GetStdHandle(STD_OUTPUT_HANDLE) y asigna las coordenadas deseadas a una estructura COORD dwPos, aplicando la reubicacion inmediata a traves de SetConsoleCursorPosition(hCon, dwPos). Esta tecnica reemplaza la invocacion ineficiente de system("cls") durante el bucle de juego, eliminando el parpadeo de pantalla y optimizando el rendimiento grafico.

Por su parte, la rutina ocultarCursor() recupera la informacion de configuracion del cursor mediante la estructura CONSOLE_CURSOR_INFO cci, estableciendo su miembro booleano bVisible en FALSE antes de invocar SetConsoleCursorInfo(hCon, &cci). De este modo, se suprime el cursor intermitente de la terminal, otorgando a la consola la apariencia de una pantalla grafica pulida.

Arquitectura de la Clase Principal: class Nave

La clase Nave constituye el nucleo del diseno orientado a objetos. Encapsula de forma estricta los atributos privados de la nave defensora y expone metodos publicos disenados con precision quirurgica para gobernar su ciclo de vida. En la Tabla 1 se describe la interfaz publica y privada de dicha clase.

Clases Complementarias del Motor: Asteroide y Bala

El motor implementa dos clases adicionales para modularizar las amenazas y los proyectiles.

La clase Asteroide gestiona las coordenadas privadas (x, y) de cada masa rocosa, automatizando su traslacion vertical descendente mediante mover(). Cuando la coordenada y supera el valor umbral de 23 (borde inferior), la clase regenera de manera autonoma su

posicion en la fila superior (y = 4) y asigna una columna aleatoria mediante rand() % 70 + 2. Por su parte, la clase Bala encapsula el comportamiento balistico de los proyectiles disparados desde el canon de la nave defensora (instanciados en x + 2, y - 1). Su metodo mover() borra el rastro anterior, decrementa la coordenada y y renderiza el glifo vertical |. Adicionalmente, el metodo booleano fueraDeRango() evalua la condicion y < 4 para indicar al bucle principal el momento exacto en que el proyectil abandona el campo de accion.

Gestion Dinamica de Memoria y Prevencion de Fugas (Memory Leaks)

Una de las complejidades tecnicas cardinales del proyecto radica en la gestion rigurosa de la memoria fisica (RAM). Las balas y asteroides se almacenan en contenedores de la biblioteca estandar como vectores de punteros: std::vector<Asteroide*> y std::vector<Bala*>. Cada disparo invoca el operador new para alojar la instancia en el monticulo (heap).

Para evitar fugas de memoria criticas (memory leaks) que degradarian el rendimiento del sistema operativo en partidas prolongadas, el algoritmo de barrido inspecciona sistematicamente ambos vectores: cuando una bala impacta contra un asteroide o traspasa el borde superior de la consola, el puntero es destruido mediante el operador delete, liberando de inmediato la memoria asignada en el heap, e inmediatamente desvinculado del vector con el metodo erase().

Estructura del Repositorio Git y Normas de Calidad

El proyecto se organiza bajo una jerarquia estricta de directorios en el repositorio local juegodenaves/, garantizando su posterior publicacion simultanea en las plataformas remotas GitHub y GitLab:

1. Raiz juegodenaves/: Alberga los archivos de configuracion .gitattributes (normalizacion de finales de linea LF), .gitignore (exclusion de binarios compilados *.exe, carpetas .vscode/ y temporales), archivo LICENSE con licencia academica MIT, y el archivo README.md que sirve de portada tecnica al repositorio.

2. Directorio src/: Contiene de forma exclusiva el codigo fuente principal main.cpp debidamente formateado, tabulado y comentado segun directrices institucionales. 3. Directorio docs/: Aloja la documentacion formal: el informe academico APA v7, el archivo CHANGELOG.md de control de versiones y el subdirectorio manuales/ con los manuales de usuario y programador.

4. Directorio capturas/: Conserva la evidencia grafica de ejecucion combate_consola.png donde se visualiza el HUD de consola con el nombre real del estudiante.

Conclusión

La arquitectura de software desarrollada para el motor de juego espacial en consola demuestra que la Programación Orientada a Objetos en C + + proporciona una plataforma inigualable para la construcción de sistemas eficientes, mantenibles y robustos. La encapsulación de variables críticas dentro del ámbito privado de la clase Nave previene categóricamente los fallos de estado derivados de accesos no controlados, garantizando la estabilidad operativa del bucle principal de juego.

Asimismo, la rigurosa administración del ciclo de vida de los objetos dinámicos mediante los operadores new y delete reafirma la necesidad de un control disciplinado de la memoria física en aplicaciones de tiempo real. Este ejercicio integral, respaldado por la elaboración de manuales técnicos normalizados bajo las normas APA 7ma edición, consolida las capacidades profesionales de diseño, codificación y documentación indispensables en la formación técnica de nivel secundario orientada a la industria informática.

Referencias

Luca Di Guglielmo

Github: https://github.com/specIux/actividad-8-lpr

Gitlab: https://gitlab.com/specIux/actividad-8-lpr#

Laboratorio de Programación 5°3°

ACTIVIDAD 8: PROGRAMACIÓN ORIENTADA A OBJETOS (POO) EN …